Montran Internship Final Exam

The purpose of this test is to assess your ability to solve programming problems. Write your solutions in Java 8. To manage properly the code and dependencies, you could use gradle to assemble and execute your component.

For the problem below, write the simplest, clearest solution you can, in the form of a short program/functions. If you don't have enough time to develop your solution, describe the architecture by using proper class skeletons and documentation.

You do not need to do any I/O, i.e., you can hard-code your input data to test the component. Keep it simple! We are primarily interested in what you write in the body of the function and architecture. However, please be sure that your solution will work for all valid input data.

The clock is ticking now, so you don't have time to ask for clarifications on any of the questions. If something is not clear to you, resolve it yourself and state in a comment in the program what was unclear and how you resolved it.

Credit Score Management System

Design and develop a Credit Score Management System that enables users to manage personal information, track credit history, calculate credit scores based on specific rules, and assess credit risk levels. The system should provide functionalities to add, edit, and delete user and credit history records, calculate credit scores dynamically, and notify users of significant changes.

1. Logic Requirements

1.1. User Management

- Add, edit, and delete user records.
- Each user should have details like name, social security number (SSN), address, and credit score.
- The SSN should be unique for each user.

1.2. Credit History Management

- Track users' credit history including details like date, type of transaction (loan, credit card, mortgage), amount, and status (paid, defaulted).
- Add, edit, and delete credit history records for each user.

1.3. Credit Score Calculation

- Implement a module to calculate the credit score based on users' credit history.
- The credit score should be influenced by factors such as the total amount of credit used, the number of defaulted transactions, and the length of credit history.
- Provide a method to update the credit score periodically.
- Ensure the credit score is calculated based on the following configurable rules:
    - Maximum credit score: 100
    - Credit Utilization (max 30 points): (Credit used / Total credit limit) * 30

- Payment History (max 35 points): (Number of on-time payments / Total payments) * 35
- Credit Age (max 15 points): (Average age of credit accounts in years) / 10 * 15
- Types of Credit (max 10 points): (Number of different types of credit) * 2 (up to 10 points)
- Recent Credit Inquiries (max 10 points): (Number of inquiries in last 2 years) * -2 (up to -10 points)

1.4. Risk Assessment

- Implement a risk assessment module that categorizes users based on their credit score into risk levels (low, medium, high).
- The module should update risk levels automatically whenever the credit score is recalculated.
- Configurable thresholds for risk levels:
  - Low Risk: Credit score >= 75
  - Medium Risk: 50 <= Credit score < 75
  - High Risk: Credit score < 50

1.5. Notifications

Notify users via email or SMS when there are significant changes to their credit score or risk level (no actual implementation of email/SMS sending required, just the logic).

1.6. Thread safety

Ensure thread safety for concurrent updates to user records and credit history.

1.7. Persistence

- Persist all data to XML with an option for JSON persistence.
- Allow for future implementation of database persistence.

1.8. Pluggable Persistence

Design the persistence mechanism to allow for easy switching between XML and JSON.

2. Required Component Algorithms

2.1. Credit Score Calculation Algorithm

- Define a formula for credit score calculation considering factors such as total credit amount, defaults, and history length.
- Ensure the formula is flexible to allow for future modifications.

2.2. Risk Assessment Algorithm

- Define thresholds for categorizing users into different risk levels based on their credit score.
- Implement logic to update risk levels automatically.

2.3. Thread Safety

Considering that the component could be used for in a multi-threaded environment, the component needs to be made thread safe. Make suer your implementation works in a multi-threaded environment and justify in the documentation what measures have you taken to make sure that is accomplished.

2.4. Notification System

Implement a notification system that triggers on significant changes to a user's credit score or risk level.

2.5. Customizable Score Weighting (Extra 2 points)

Allow administrators to adjust the weighting of different factors (e.g., credit utilization, payment history, credit age) in the credit score calculation. You could let the admin use a settings file to configure these weights.

3. Demo section

Provide a main method demonstrating the following:

- Adding, editing, and deleting user records.
- Adding and updating credit history.
- Calculating and updating credit scores.
- Categorizing users into risk levels.
- Triggering notifications on significant changes.

4. Examples

Here are five sample scenarios demonstrating the credit score calculation based on the given rules. Each scenario will include details about the user's credit history and the resulting credit score.

Scenario 1: Excellent Credit History

User Credit History:

Credit Utilization: Total credit limit $10,000, Credit used $2,000 (20%)
Payment History: 20 payments, all on-time (100%)
Credit Age: Average age of accounts 5 years
Types of Credit: 3 different types (credit card, mortgage, auto loan)
Recent Credit Inquiries: 1 inquiry in the last 2 years

Calculation:

Credit Utilization (max 30 points): (2000 / 10000) * 30 = 6 points
Payment History (max 35 points): (20 / 20) * 35 = 35 points
Credit Age (max 15 points): (5 / 10) * 15 = 7.5 points
Types of Credit (max 10 points): 3 * 2 = 6 points
Recent Credit Inquiries (max -10 points): 1 * -2 = -2 points
Total Credit Score: 6 + 35 + 7.5 + 6 - 2 = 52.5

Scenario 2: Good Credit History with a Few Late Payments

User Credit History:

Credit Utilization: Total credit limit $20,000, Credit used $8,000 (40%)
Payment History: 15 payments, 13 on-time, 2 late (87%)
Credit Age: Average age of accounts 8 years
Types of Credit: 4 different types (credit card, mortgage, auto loan, student loan)
Recent Credit Inquiries: 0 inquiries in the last 2 years

Calculation:

Credit Utilization (max 30 points): (8000 / 20000) * 30 = 12 points

Payment History (max 35 points): (13 / 15) * 35 = 30.33 points
Credit Age (max 15 points): (8 / 10) * 15 = 12 points
Types of Credit (max 10 points): 4 * 2 = 8 points
Recent Credit Inquiries (max -10 points): 0 * -2 = 0 points
Total Credit Score: 12 + 30.33 + 12 + 8 + 0 = 62.33

Scenario 3: Moderate Credit History with High Utilization

User Credit History:
Credit Utilization: Total credit limit $5,000, Credit used $4,500 (90%)
Payment History: 10 payments, 9 on-time, 1 late (90%)
Credit Age: Average age of accounts 3 years
Types of Credit: 2 different types (credit card, auto loan)
Recent Credit Inquiries: 3 inquiries in the last 2 years

Calculation:
Credit Utilization (max 30 points): (4500 / 5000) * 30 = 27 points
Payment History (max 35 points): (9 / 10) * 35 = 31.5 points
Credit Age (max 15 points): (3 / 10) * 15 = 4.5 points
Types of Credit (max 10 points): 2 * 2 = 4 points
Recent Credit Inquiries (max -10 points): 3 * -2 = -6 points
Total Credit Score: 27 + 31.5 + 4.5 + 4 - 6 = 61

Scenario 4: Poor Credit History with Multiple Late Payments

User Credit History:
Credit Utilization: Total credit limit $15,000, Credit used $12,000 (80%)
Payment History: 12 payments, 6 on-time, 6 late (50%)
Credit Age: Average age of accounts 2 years
Types of Credit: 1 type (credit card)
Recent Credit Inquiries: 5 inquiries in the last 2 years

Calculation:
Credit Utilization (max 30 points): (12000 / 15000) * 30 = 24 points
Payment History (max 35 points): (6 / 12) * 35 = 17.5 points
Credit Age (max 15 points): (2 / 10) * 15 = 3 points
Types of Credit (max 10 points): 1 * 2 = 2 points
Recent Credit Inquiries (max -10 points): 5 * -2 = -10 points
Total Credit Score: 24 + 17.5 + 3 + 2 - 10 = 36.5

Scenario 5: New User with No Credit History

User Credit History:
Credit Utilization: Total credit limit $1,000, Credit used $100 (10%)
Payment History: 1 payment, on-time (100%)
Credit Age: Average age of accounts 0.1 years
Types of Credit: 1 type (credit card)
Recent Credit Inquiries: 1 inquiry in the last 2 years

Calculation:

Credit Utilization (max 30 points): (100 / 1000) * 30 = 3 points
Payment History (max 35 points): (1 / 1) * 35 = 35 points
Credit Age (max 15 points): (0.1 / 10) * 15 = 0.15 points
Types of Credit (max 10 points): 1 * 2 = 2 points
Recent Credit Inquiries (max -10 points): 1 * -2 = -2 points
Total Credit Score: 3 + 35 + 0.15 + 2 - 2 = 38.15

These scenarios demonstrate how different aspects of a user's credit history impact their overall credit score calculation.

5. Notes and Advices

- ✓ Start coding as soon as possible and don't spend hours in reading the requirements document.
- ✓ Identify the basic entities of the design, manager classes and the extension points for the component, so that the component will be clean and flexible.
- ✓ Quickly create a simple, working core and then expand later with more functionality, as required. It is better to have a working, unfinished component, than just an unfinished one.
- ✓ The component's architecture should be modular and allow easily extending it by plugging in additional functionality.
- ✓ Identify and use correctly design patterns, while keeping the solution simple and clean.
- ✓ Document your classes and methods. Commenting in one/two lines to catch the essence is enough. Empty comments that do not bring any value to your component, like "this is class X", should be avoided as they just waste everybody's time.
- ✓ Do not waste time on small things like documenting getters or setters.
- ✓ Use custom exceptions wisely and reuse system exceptions where possible.
- ✓ Pay attention to performance – both from the memory usage point of view, as well as the complexity of the algorithms and required time for execution.